

Experiment 13 – Robot Framework Basics and Browser Automation

1. Module Name and Experiment Title

Module Name: Module 4 – Robot Framework

Experiment Number: 13

Experiment Title: Robot Framework Basics and Browser Automation

Programming Language: Python (Robot Framework)

Website: The Internet – Login Page

Website URL: <https://the-internet.herokuapp.com/login>

2. Problem Statement

To implement basic browser automation using Robot Framework and SeleniumLibrary. The experiment involves opening a web browser, navigating to a login page, verifying the presence of login fields, entering valid credentials, and checking whether the login operation is successful.

The automation script should execute the test cases and generate execution reports and logs to verify the results.

3. Objective

The main objectives of this experiment are:

1. To understand the basic syntax and structure of Robot Framework.
 2. To configure SeleniumLibrary for browser automation.
 3. To automate browser operations using Robot Framework keywords.
 4. To locate and interact with web elements using SeleniumLibrary.
 5. To verify the presence of login form elements.
 6. To automate the login process using valid credentials.
 7. To use setup and teardown keywords for browser management.
 8. To generate execution logs and HTML reports.
-

4. Tools, Software and Concepts Used

Tools and Software

Tool/Software	Purpose
Python	Programming language used to execute the automation framework
Robot Framework	Keyword-driven test automation framework
SeleniumLibrary	Library used for browser automation
Google Chrome	Browser used for testing
Visual Studio Code	Code editor used to write and execute the test script
PowerShell	Command-line interface for installing dependencies and running tests

Concepts Used

- Robot Framework test cases and keywords
- SeleniumLibrary
- Browser automation
- Element identification using ID and CSS selectors
- Suite Setup and Suite Teardown
- Assertions and verification
- Screenshot capturing
- HTML test reports and execution logs

5. Source Code / Implementation Steps

Step 1: Install the Required Libraries

The following dependencies were installed using `pip`:

```
robotframework
robotframework-seleniumlibrary
```

The dependencies were saved in `requirements.txt` and installed using:

```
pip install -r source_code/requirements.txt
```

Step 2: Create the Project Structure

The following folder structure was used for the experiment:

```
Experiment 13 - Robot Framework Basics and Browser Automation
├── source_code
│   ├── test_browser.robot
│   └── requirements.txt
├── screenshots
├── Report_Word
└── reports
    └── output.xml
```

```
└─ log.html
└─ report.html
```

Step 3: Write the Robot Framework Script

The `test_browser.robot` file contains the test cases and reusable keywords required to automate the login process.

Source Code:

```
*** Settings ***
Library      SeleniumLibrary

Suite Setup      Open Test Browser
Suite Teardown   Close Browser

*** Variables ***
${URL}           https://the-internet.herokuapp.com/login
${USERNAME}      tomsmith
${PASSWORD}      SuperSecretPassword!
${BROWSER}       chrome

*** Test Cases ***
Verify Login Page
    [Documentation]    Verify that the login page opens successfully.
    Open Login Page
    Page Should Contain Element    id=username
    Page Should Contain Element    id=password
    Capture Page Screenshot        screenshots/01_login_page.png
    Log    Login page displayed successfully.

Verify Successful Login
    [Documentation]    Verify login using valid credentials.
    Open Login Page
    Input Text    id=username    ${USERNAME}
    Input Text    id=password    ${PASSWORD}
    Capture Page Screenshot        screenshots/02_credentials_entered.png
    Click Button    css=button[type="submit"]
    Wait Until Page Contains    You logged into a secure area!
    timeout=10s
    Page Should Contain    You logged into a secure area!
    Capture Page Screenshot        screenshots/03_successful_login.png
    Log    Login successful and verified.

*** Keywords ***
Open Test Browser
    Open Browser    about:blank    ${BROWSER}
    Maximize Browser Window

Open Login Page
    Go To    ${URL}
    Wait Until Element Is Visible    id=username    timeout=10s
```

Step 4: Execute the Test Script

The following command was used to execute the Robot Framework test cases and generate the reports:

```
robot --outputdir reports source_code/test_browser.robot
```

The command runs the test cases defined in `test_browser.robot` and saves the execution results in the `reports` directory.

Step 5: Test Case Description

Test Case 1: Verify Login Page

- Opens the Chrome browser.
- Navigates to the login page.
- Waits until the username field becomes visible.
- Verifies the presence of the username and password fields.
- Captures a screenshot of the login page.

Test Case 2: Verify Successful Login

- Navigates to the login page.
- Enters the valid username and password.
- Captures a screenshot showing the entered credentials.
- Clicks the Login button.
- Waits for the successful login message.
- Verifies that the success message is displayed.
- Captures a screenshot of the secure area.

Step 6: Generate Execution Reports

Robot Framework automatically generated the following files after execution:

- **output.xml:** Contains the structured test execution results.
- **log.html:** Displays detailed execution logs, including individual keywords and their statuses.
- **report.html:** Provides a summary of the test execution, including the number of passed and failed tests.

6. Output with Clearly Labelled Screenshots

The experiment was executed successfully using Robot Framework and SeleniumLibrary. Both test cases passed without any failures.

Screenshot 1: Login Page

Description: The login page of The Internet website was opened successfully in Google Chrome. The username and password fields were displayed.

Login Page

This is where you can log into the secure area. Enter *tomsmith* for the username and *SuperSecretPassword!* for the password. If the information is wrong you should see error messages.

Username

Password

 Login

Powered by [Elemental Selenium](#)

For more on CuiLab

Screenshot 2: Credentials Entered

Description: The automation script entered the valid username and password into the respective fields before clicking the Login button.

Login Page

This is where you can log into the secure area. Enter *tomsmith* for the username and *SuperSecretPassword!* for the password. If the information is wrong you should see error messages.

Username

Password

 Login

Powered by [Elemental Selenium](#)

For more on CuiLab

Screenshot 3: Successful Login

Description: After submitting the valid credentials, the website displayed the successful login message and opened the Secure Area page.

You logged into a secure area!

Post me on GitHub

Secure Area

Welcome to the Secure Area. When you are done click logout below.

Logout

Powered by [Elemental Selenium](#)

Screenshot 4: Robot Framework Execution Log

Description: The Robot Framework execution log shows that both test cases passed successfully. It also displays the individual keywords executed during the tests.

The screenshot displays a Robot Framework execution log report in a web browser. The report is titled "Test Browser Log" and was generated on 20260927 at 22:02:40 UTC+05:30, 3 minutes and 43 seconds ago. It includes a "REPORT" button in the top right corner.

Statistics

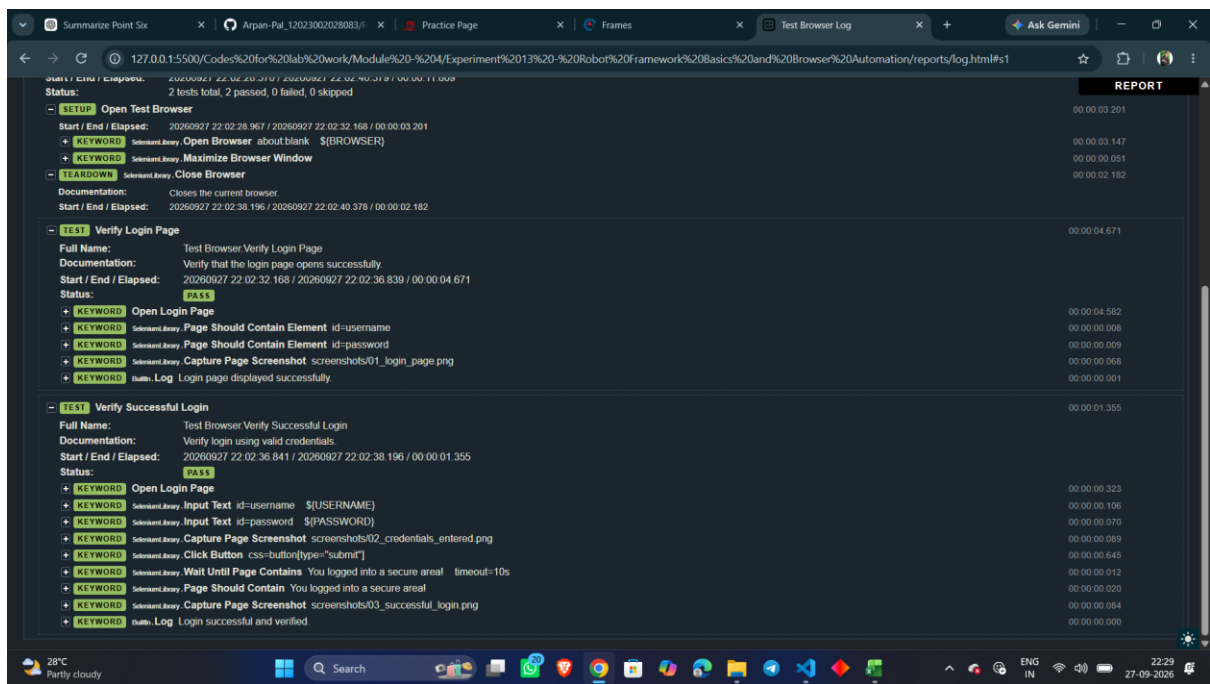
Total Statistics		Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
All Tests		2	2	0	0	00:00:06	2 / 0 / 0

Statistics by Tag		Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
No Tags		0	0	0	0	00:00:00	0 / 0 / 0

Statistics by Suite		Total	Pass	Fail	Skip	Elapsed	Pass / Fail / Skip
Test Browser		2	2	0	0	00:00:12	2 / 0 / 0

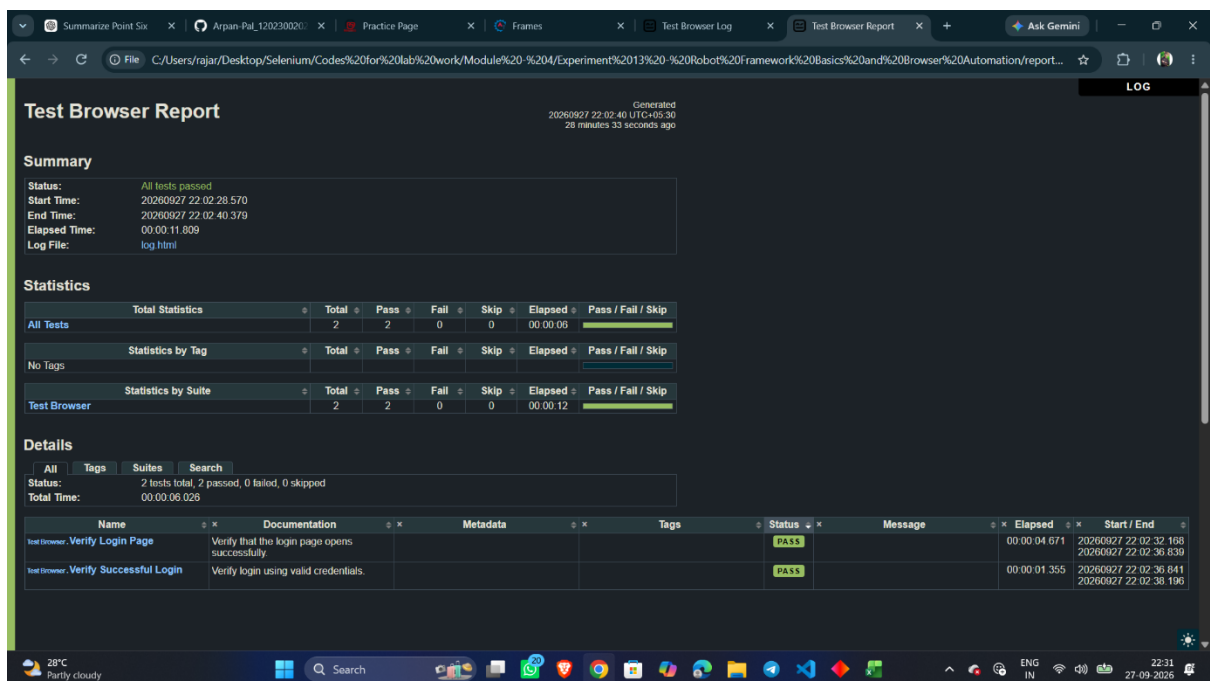
Execution Log

- SUITE: Test Browser** (00:00:11.809)
 - Full Name: Test Browser
 - Source: C:\Users\vajan\Desktop\Selenium\Codes for lab work\Module - 4\Experiment 13 - Robot Framework Basics and Browser Automation\source_code\test_browser.robot
 - Start / End / Elapsed: 20260927 22:02:28.570 / 20260927 22:02:40.379 / 00:00:11.809
 - Status: 2 tests total, 2 passed, 0 failed, 0 skipped
 - SETUP: Open Test Browser** (00:00:03.201)
 - Start / End / Elapsed: 20260927 22:02:28.967 / 20260927 22:02:32.168 / 00:00:03.201
 - KEYWORD: SeleniumLibrary.Open Browser** about:blank \${BROWSER} (00:00:03.147)
 - KEYWORD: SeleniumLibrary.Maximize Browser Window** (00:00:00.051)
 - TEARDOWN: SeleniumLibrary.Close Browser** (00:00:02.182)
 - Documentation: Closes the current browser.
 - Start / End / Elapsed: 20260927 22:02:38.196 / 20260927 22:02:40.378 / 00:00:02.182
 - TEST: Verify Login Page** (00:00:04.671)
 - Full Name: Test Browser Verify Login Page
 - Documentation: Verify that the login page opens successfully.
 - Start / End / Elapsed: 20260927 22:02:32.168 / 20260927 22:02:36.839 / 00:00:04.671
 - Status: **PASS**
 - KEYWORD: SeleniumLibrary.Open Login Page** (00:00:04.582)
 - KEYWORD: SeleniumLibrary.Page Should Contain Element** id=username (00:00:00.008)



Screenshot 5: Robot Framework Test Report

Description: The HTML report displays the overall test execution summary, including the total number of tests, passed tests, failed tests, and execution status.



Test Execution Summary

Parameter	Result
Total Test Cases	2
Passed	2

Parameter	Result
Failed	0
Skipped	0
Overall Status	PASS
Execution Time	Approximately 12 seconds

7. Result, Observation and Conclusion

Result

The experiment was successfully completed using Robot Framework and SeleniumLibrary. Two automated test cases were executed on the login page of The Internet website.

Both test cases passed successfully, with zero failures and zero skipped tests. The login page was verified, valid credentials were entered, and the successful login message was confirmed.

Observation

1. Robot Framework successfully executed the browser automation script.
2. SeleniumLibrary provided the required browser interaction keywords.
3. The login page elements were located and verified successfully.
4. The automation script successfully entered the valid credentials.
5. The login operation completed successfully, and the expected confirmation message was displayed.
6. Robot Framework generated the HTML execution log and report.

Conclusion

This experiment demonstrated the use of Robot Framework for basic web browser automation. It provided practical experience with test cases, reusable keywords, SeleniumLibrary, browser setup and teardown, element verification, and automated login testing.

The successful execution of both test cases confirmed that the implemented automation script worked as expected. The generated HTML report and execution log provided a convenient way to review and verify the test results.